Distributed Systems Programming

A.Y. 2026/27

Laboratory 1

The two main topics that are addressed in this laboratory activity are:

- design of JSON schemas;
- design of REST APIs.

In order to have a complete experience, an implementation of the designed REST APIs will also be developed, by connecting the designed APIs to an already existing implementation of the service.

The context in which this laboratory activity is carried out is a *Film Manager* application, where users can keep track of the films they have watched and the reviews that they have written for them.

The tools that are recommended for the development of the solution are:

- *Visual Studio Code* (https://code.visualstudio.com/) for the validation of JSON files against the schemas, and for the implementation of the REST APIs;
- *OpenAPI (Swagger) Editor*, an extension for Visual Studio Code, for the design and editing of the REST APIs in OpenAPI;
- *OpenAPI Generator* (https://openapi-generator.tech/) for the automatic generation of the server stub from the OpenAPI specification;
- *PostMan* (https://www.postman.com/) for testing the REST APIs implementation;
- *DB Browser for SQLite* (https://sqlitebrowser.org/) for managing the database.

The provided implementation requires Node.js 22, npm, and Java. On the LabInf machines, Node.js 22 can be installed using Node Version Manager (nvm) and selected by running “nvm install 22”.
The Javascript language and the Express (https://www.npmjs.com/package/express) framework are used for developing the implementation of the RESTful web service**.**

1. Design of JSON schemas

The first activity is about the design of JSON schemas for three core data structures of the *Film Manager*, i.e., the *users* who want to manage their film lists by means of this application, the *films* they have watched and/or reviewed, and the *review invitations* that

a user may issue to another user. All the design choices for which there are no specific indications are left to the students.

A *user* data structure is made of the following fields:

- *id*: unique identifier of the user data structure in the *Film Manager* platform (mandatory);
- *name*: username of the user;
- *email*: email address of the user, which must be used for the authentication to the platform (mandatory, it must be a valid email address);
- *password*: the user's password, which must be used for the authentication to the platform (the password must be at least 6 characters long and at most 20 characters long).

A *film* data structure is made of the following fields:

- *id*: unique identifier of the *film* data structure in the *Film Manager* platform (mandatory);
- *title*: textual title of the film (mandatory);
- *owner:* the id of the film data structure owner, i.e., the user who created it (mandatory);
- *private*: a Boolean property, set to true if the *film* data structure is marked as private, false if it is public (mandatory). A *film* data structure is said private if only its owner can access it, public if every user can access it;
- *watchDate*: the date when the film was watched by the owner, expressed in the YYYY-MM-DD format (YYYY is the year, MM is the month, DD is the day). This property can be included in the *film* data structure only if *private* is true;
- *rating*: a non-negative integer number (maximum 10) expressing the rating the owner has given to the film. This property can be included in the *film* data structure only if *private* is true;
- *favorite*: a Boolean property, set to true if that film is among the favourite ones of the user, false otherwise (default value: false). This property can be included in the *film* data structure only if *private* is true.

A *review* data structure is made of the following fields:

- *filmId*: unique identifier of the film for which a review invitation has been issued (mandatory);
- *reviewerId*: unique identifier of the user who has received the review invitation (mandatory);

- *completed*: a Boolean property, set to true if the review has been completed, false otherwise (mandatory)*;*
- *reviewDate*: the date when the review has been completed by the invited user, expressed as a string in the YYYY-MM-DD format (YYYY is the year, MM is the month, DD is the day). This property can be included only if *completed* is true, and in that case it is mandatory;
- *rating*: a non-negative integer number (maximum 10) expressing the rating of the review the user has completed. This property can be included only if *completed* is true, and in that case it is mandatory;
- *review*: a textual description of the review (it must be no longer than 1000 characters). This property can be included only if *completed* is true, and in that case it is mandatory

The JSON Schema standard that must be used for this activity is the Draft 7 (http://json-schema.org/draft-07/schema#).

After completing the design of the schemas, it is suggested to write some JSON files as examples, and to validate them against the schemas in Visual Studio Code. In this development environment, validation errors are shown in the editor and in the "Problem" view. You can access this view in two alternative ways:

- following the path View -> Problems;
- pressing Ctrl+Shift +M.

2. Design and implementation of REST APIs

The second lab activity is about the design and implementation of REST APIs for the Film Manager platform.
Section 2.1 provides the design specifications, while Section 2.2 provides concrete instructions and guidelines for developing the design and implementation.

2.1 REST APIs Design Specifications

The *Film Manager* platform allows users to track information about the films they have watched, or for which they want to issue a review invitation or for which they must perform a review. Two key concepts are film *owner* and film *reviewer*. The former is the user who creates the *film* data structure on the platform; the latter is a user who is in

charge of reviewing the film after being invited by the owner to do so. The platform maintains information about the users who are authorized to use it and their films in a database. Each user is authenticated by means of a personal password, which, as common practice recommends, is not stored in the database as plain text. Instead, a salted hash of the password is computed and stored in the database. The salt must be random and at least 16 bytes long.

Most of the features of the platform can be accessed only by authenticated users. The only operations that can be used by anyone without authentication are:

- the authentication operation itself (login);
- the operation to retrieve the list of all the *film* elements that are marked as public;
- the operation to retrieve a single public *film* element;
- the operation to retrieve the list of the reviews of a public *film*;
- the operation to retrieve a single review of a public *film;*
- the operation to retrieve the Film Manager entry-point resource.

For authentication, the user sends their email and password to the platform, which checks if these credentials are correct. Emails used for authentication must be unique on the platform. The authentication mechanism that is used by the platform is a session-based authentication, where the session data are saved server-side, and the client receives a session ID in a cookie. The management of this session-based authentication mechanism can be implemented by using the Passport middleware (http://www.passportjs.org/) and the *express-session* module (https://www.npmjs.com/package/express-session). For this authentication mechanism and authenticated communication to be secure enough, it would be necessary that the REST APIs are made available on HTTPS only. However, in the implementation produced for this Lab, which is not for production, we will expose the APIs on HTTP to facilitate debugging.

An authenticated user has access to the CRUD operations for the *film* elements:

- The user can create a new film. If the creation of the film is successful, the platform assigns it a unique identifier. The creator of the film becomes its owner. The film review invitation to the users is explained later in this document.
- The user can retrieve a single existing film, if one of the following conditions is satisfied:
    1) the film is marked as public;
    2) the user is the owner of the film.

The user can also retrieve the list of all the films that she created, and the list of all the films that she has been invited to review.

- The user can update an existing film if she is the owner of the film. However, this operation does not allow changing its visibility from public to private (and vice versa).
- The user can delete an existing film, if she is the owner of the film.

A central feature of the *Film Manager* platform is issuing invitations to review public films to the users who have access to the platform. The main operations related to this feature are:

- The owner of a public film can issue a review invitation to a user (the reviewer may be the owner herself).
- The owner of a film can remove a review invitation if the review has not yet been completed by the reviewer.
- A reviewer invited for a film can mark the review as completed, also updating the review date, the rating and the textual description of the review.

Review invitations for a film can be issued to multiple users at the same time. Each user who has received a review invitation can mark their review for that film as completed.

Finally, the platform must provide an additional operation that automatically issues review invitations for all films without any existing invitations, ensuring that the invitations are distributed evenly among users (so that each user receives approximately the same number of invitations). While the design of this feature is mandatory, its implementation is not required.

Here are some recommendations for the design and development of the REST APIs:

- When a list of films is retrieved, a pagination mechanism is recommended, to limit the size of the messages the platform sends back.
- When the users send a JSON *film* element to the platform (e.g., for the creation of the film in the database, or for the update of an existing film), this input piece of data should be validated against the corresponding JSON schema.
- The platform should be HATEOAS (Hypermedia As The Engine of Application State) compliant. Hyperlinks should be included in responses to enable link-based navigation and features should be self-describing. For example, when the *Film Manager* platform sends back a JSON *film* or *user* object, this should include a *self* link referring to the URI where the resource can be retrieved by a GET operation.

Moreover, a client should be able to perform all operations without having to build URIs.

2.2 Instructions and Guidelines for developing the solution

The design of the REST APIs will be specified and documented in an OpenAPI specification, while the implementation will be developed in Node.js using the Express framework.
For the design, you should reuse the JSON Schemas developed in the first part of the assignment, customizing them for being used in the REST APIs. The resulting OpenAPI document will then be used to automatically generate the server stub, so ensuring alignment between the specification and the implementation.

For this activity, we provide an already configured project, which also includes material useful for development. It is available in the Lab01 GitHub repository in the polito-dsp-2026-2027 organization (you will have access to the repository starting from the second session of the laboratory activity, which is focused on the REST APIs design and implementation). You should use this project as a starting point, rather than creating a separate project. You can find a guide for developing this second part of the lab in the README file. Here, some further details are provided.

Specifically, the project includes a script to run the generator using a set of custom templates that will produce code compliant with the code organization described below. To execute it, run `npm run generate` from the root directory of the project.

Since the focus of this course is on the design of REST APIs, and not on the implementation of the application logic, the implementation of most of the necessary functionalities is made available as part of the laboratory material.

The implementation provided for this laboratory is already organized as a complete project. Therefore, it is not necessary to generate a separate project and subsequently copy the provided files into it.

The project is a skeleton of the target software, containing:

- the “index.js” file, containing the configuration of the Express server, including CORS, session management, Passport authentication and OpenAPI-based routing;
- the “controllers/validator.js” file, containing the configuration of AJV and the validation functions initialized from the JSON Schemas;

- the “passport-config.js” file, containing the configuration of the Passport middleware;
- the “package.json” file, containing the required dependencies and the commands used to generate and run the server;
- the “database/” folder, containing the SQLite database with preconfigured tables and data;
- the “components/” folder, containing the object definitions of the main resources
- the “controllers/validator.js”, containing the validators needed to check the controller function request against the corresponding json schema
- the “controllers/link.js”, containing some helper functions that can be used to add additional links, like selflinks, to the controller functions’ response;
- the “service/” folder, containing the documented implementation of several functions providing the application logic;
- the “openapi-templates/” folder and the “openapitools.json” file, containing the configuration used for the automatic generation of the server stub;
- the “api/openapi.json” file, where the OpenAPI specification must be written;
- the “json-schemas” folder, where to insert the json schema produced;
- the “database” folder containing the SQLite database that will be used for the application;
- the “utils” folder, containing additional files needed to make the application run.

The parts that must be completed during the laboratory activity are the JSON Schemas in the “/json-schemas” folder, the OpenAPI specification in the “/api” folder, the resource links defined in “/controllers/link.js”, and the REST controllers in the “/controllers” folder.

The development activity should proceed as follows:

1. include the JSON Schemas describing the main data structures of the Film Manager platform into the “json-schemas” folder;
2. complete the OpenAPI specification contained in the “api/openapi.json” file, which is provided as an initial skeleton, defining the resources (using a tag for each resource), operations, with related parameters, request and response bodies, exit codes, etc., and security requirements;
3. generate the server stub by running the “npm run generate” command from the root directory of the project; the generated files will be placed in the “generated/” folder. They provide the routing layer that dispatches the requests of the operations defined in the OpenAPI document to the application controllers. The contents of the “generated/” folder must not be edited

manually, since they are recreated each time the generation command is re-executed (e.g., after an update of the API specification).
4. replace all occurrences of the "/change/me" placeholder with the correct API URIs and complete the helper functions in the controllers/links.js file;
5. implement the controllers in the "controllers/" folder, using the functions provided by the service layer when appropriate.

Whenever the OpenAPI specification is modified, the "npm run generate" command must be executed again before running the server.

Routing

Routing refers to determining how an application responds to a client request directed to a particular endpoint, identified by a URI and an HTTP method. In the provided implementation, routing is automatically configured according to the OpenAPI specification. Therefore, the routes exposed by the server can be modified by changing the OpenAPI specification document.

To separate route definitions from the application logic that handles the requests, the project uses a template to generate an adapter layer. This layer connects each operation defined in the OpenAPI specification to a corresponding controller function.

For each operation described in the OpenAPI document, two fields defined under the corresponding HTTP method are particularly important for connecting the API specification to its implementation correctly:

1. the first *tag* associated with the operation determines the name of the controller file, and it should also be the name of the target resource of the operation;
2. the *operationId* determines the name of the function exported by that controller.

For example, let us suppose that the GET operation associated with the "/api/films/public" path has "ApiFilmsPublic" as its first tag and "getPublicFilms" as its operationId. In this case, the corresponding implementation must be provided by the "controllers/ApiFilmsPublic.js" file, which must export the "getPublicFilms" function.

The routing adapters are automatically generated in the "generated/" folder when the "npm run generate" command is executed.

Controllers Implementation

The controllers must implement the REST-specific part of the application. They receive the client requests, validate the request data as described later in the Validation section, invoke the appropriate functions provided by the service layer, and produce the corresponding HTTP responses, including the required HATEOAS links as described later in the Links section. The service layer is independent of the communication protocol and may return error conditions as plain strings. The controllers are therefore responsible for translating the results returned by the services into the appropriate HTTP status codes and response bodies. The functions available in the “service/” folder can be used to implement the required operations. They may be modified or extended if the API design requires additional application logic.

Before sending successful responses, controllers must use the functions provided in “controllers/link.js” to add the required HATEOAS links to the returned resource representations.

The controllers in the “controllers/” folder must be implemented and are distinct from the automatically generated routing adapters contained in the “generated/” folder. Their organization and naming must follow the mapping between OpenAPI tags, controller files and operationIds described in the Routing section.

Validation

The validation of input data must be performed inside the controllers. The JSON data sent by clients must be validated according to the corresponding JSON Schemas developed in the first part of the laboratory. JSON Schema validation is performed by means of AJV. The provided controllers/validator.js module loads the JSON Schemas and compiles the corresponding validation functions that can later be used to validate the requests before running the application logic. It is also necessary to validate path parameters and query parameters when needed, checking that values coming from different parts of the request are acceptable and consistent with each other. If the input data is not valid, the controller must return an appropriate HTTP error response.

Links

To satisfy the HATEOAS requirements, the representations returned by the REST API must contain links that allow clients to navigate the API. REST links are added by the controllers when preparing HTTP responses. The provided “controllers/link.js” module contains functions that support this operation.

The getFilmManager() function returns the Film Manager entry-point representation. You must replace its /change/me placeholders with the URIs of the corresponding operations. The addToFilm(), addToUser(), and addToReview() functions must be completed so that they add the appropriate links to either an individual resource or a collection of resources. The correct URIs can be determined from the paths defined in api/openapi.json.

User authentication

User authentication with Passport and express-session is managed server-side in the following way:

1. when a user tries to authenticate to the platform, a Passport Local Strategy is used to check if the user email exists, and the specified password is correct;
2. if the authentication is successful, the platform creates an authenticated session with the *express-session* module. In the creation of this session, the following value for the three required parameters are specified:
    a. the *secret* parameter set to a (possibly random) string that is used for signing the session id cookie (named *connect.sid*);
    b. the *resave* parameter set to false;
    c. the *saveUninitialized* parameter set to false;
3. the server includes the *connect.sid* cookie in the HTTP Response and sends it back to the client (this operation is automatically managed by Passport and it is transparent to the user);
4. when a user requests an operation that requires authentication, an authentication middleware verifies if the requesting user has been previously authenticated. In this middleware, *req.isAuthenticated()* is used to verify user authentication.

Within the provided material, these aspects are already configured. Operations requiring authentication must be identified in the OpenAPI specification by associating them with the provided cookie-based security scheme (i.e., by adding the security field and referring to cookieAuth).

Database management

We provide a database using *DB Browser for SQLite*, already populated with some records, which you can use or extend for your implementation of the server. The credentials are stored in the /database/passwords_databases.txt file of the laboratory repository. If you want to investigate the database, you can use the VS code extension SQLite Viewer and open the *.db* file with the IDE.

For the hash computation, the bcrypt module of node.js is used. If you want to create a new user, you can use the following website to generate the hash of a password, according to bcrypt: https://www.browserling.com/tools/bcrypt. If the generated hash starts with $2y$, replace that part with $2b$, as the node bcrypt module does not support $2y$ hashes.